

PROJET DE FIN DE MODULE

Objectifs

Implémenter un système multi-agent soit un écosystème intelligent et interactif capable de raisonner, de planifier et d'exécuter des tâches complexes de manière autonome ou semi-autonome dans un domaine de votre choix. Dans ce projet, vous devez mettre en évidence comment on peut concevoir un tel système étape par étape.

Modalités de travail

- Travail individuel
- Le rapport doit être déposé sur Google Classroom.

Instructions

Votre travail doit contenir les points suivants :

Workflow Agentique et Orchestration : La solution doit démontrer une orchestration fluide des agents via LangChain. Vous devez justifier la méthode de collaboration choisie (séquentielle, hiérarchique) pour décomposer une mission complète.

RAG Agentique (Retrieval-Augmented Generation) ou RAG multi-corpus: Intégration d'un système de récupération de données contextuelles permettant aux agents d'accéder à des connaissances spécifiques de manière dynamique.

Humain dans la Boucle (Human-in-the-loop) : Mise en place de points de contrôle où l'intervention humaine est nécessaire pour valider une action critique de l'agent ou corriger un raisonnement, garantissant ainsi la sécurité et la fiabilité.

Bonus - Interface Utilisateur (UI Web) : Si possible, le développement d'une interface web fonctionnelle permettant d'interagir avec le système agentique en temps réel.

Livrable

Un rapport format PDF déposé sur Google Classroom avant le délai.

Le rapport doit contenir un lien vers le code (Lien publique sur GiTHUB)

Bonnes pratiques

1 – Structure de projet claire, par exemple, il faut séparer :

- agents
- tools
- graph (logique LangGraph)
- config LLM

2 – Il faut avoir une gestion de prompts. Par exemple :

```
prompts/  
├ researcher.txt  
└ supervisor.txt
```

3 – Il ne faut pas mélanger LangChain et LangGraph sans logique claire

4 – Le fichier README.md doit contenir :

- Le titre et une description du projet
- L'architecture claire du projet, par exemple :

```
## Architecture
```

- Supervisor: orchestrates the workflow
- Researcher: gathers information
- Coder: generates code

- Exigences techniques, par exemple :

```
## Tech Stack
```

- LangChain
- LangGraph
- uv (dependency management)
- Ollama / OpenAI

- Installation

```
## Installation
```

```
```bash
```

```
git clone ...
```

```
cd project
```

```
uv venv
```

```
uv sync
```

```
5) Configuration
```

```
```md
```

```
## Configuration
```

Create a `.env` file:

```
OPENAI_API_KEY=...
```

- Structure du projet
- Tests

5- Reproductibilité des environnements avec uv

- uv lock : génère ou met à jour uv.lock

uv.lock est un fichier de verrouillage des dépendances (lock file)

- uv sync :

Crée / met à jour l'environnement virtuel

Installe toutes les dépendances

Respecte strictement le fichier uv.lock

RAG multi-corpus

Le RAG multi-corpus consiste à utiliser plusieurs bases de connaissances spécialisées pour répondre à une question. Les stratégies principales de ce concept sont :

Routing (sélection d'un corpus) : Le système choisit un seul corpus

Exemple :

- Pour une question médicale utiliser la base médicale
- Pour une question juridique utiliser la base juridique

Multi-retrieval (parallèle) : On interroge plusieurs corpus en même temps puis on fusionne ou on tri (reranking)

Approche hybride : une combinaison des deux

Avec Langchain le RAG multi-corpus peut se faire avec la méthode `create_agent` vu en cours

- Une fonction `retrieve` par corpus
- Chaque fonction `retrieve` dans un tool
- L'agent choisit le tool

C'est un routing implicite par LLM car l'agent :

1. Analyse la question
2. Choisit un tool (`search_science` ou `search_legal`)
3. Récupère les documents
4. Génère une réponse

Nous pouvons réaliser une extension avec LangGraph

- node router
- node retriever
- node generator